

JAVASCRIPT DEEP DIVE

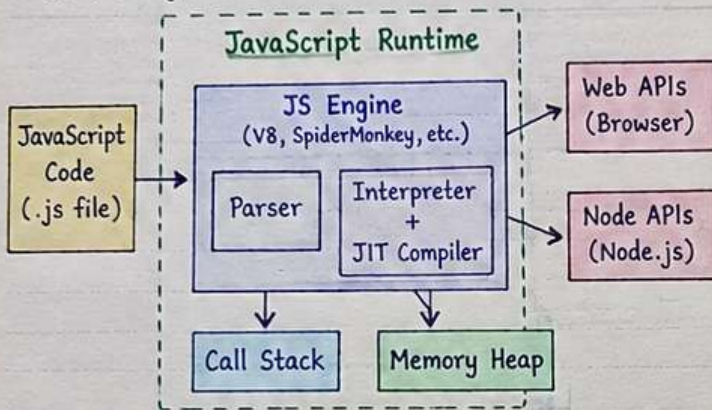
50 Concepts You Must Know

"Small Steps
Big Skills"

Page 1 : JavaScript Fundamentals

1 JavaScript Runtime

JavaScript is a high-level, single-threaded, interpreted (JIT compiled) language that runs in a runtime environment like a browser or Node.js.



Key Points:

- JS is single-threaded (one call stack).
- Uses event loop for async operations.
- Runs in browser (with Web APIs) or Node.js (with Node APIs).
- JIT (Just In Time) compilation makes it faster.

2 var, let, const

These are used to declare variables in JavaScript.

Feature	var	let	const
Scope	Function	Block	Block
Re-declare	✓ Allowed	✗ Not allowed	✗ Not allowed
Re-assign	✓ Allowed	✓ Allowed	✗ Not allowed
Hoisting	✓ Yes (initialized as undefined)	✓ Yes (but in TDZ)	✓ Yes (but in TDZ)

Example:

```
var x = 10;
var x = 20; // allowed
console.log(x); // 20
```

```
let y = 10;
// let y = 20; // Error
y = 20; // allowed
console.log(y); // 20
const z = 10;
// z = 20; // Error
console.log(z); // 10
```

Note:

- Prefer let and const over var.
- Use const by default, unless you need to reassign.
- var can lead to unexpected behaviour due to function scope.

3 Primitive vs Reference Types

JavaScript has two types of data types:

- Primitive Types → store actual value
- Reference Types → store a reference (address) to the value in memory.

Primitive Types (7)	Reference Types
• String • Number • Boolean • Undefined • Null • Symbol (ES6) • BigInt (ES2020)	• Object • Array • Function • Date • RegExp • Map, Set • and more...

Example:

```
// Primitive (copy of value)
let a = 10;
let b = a;
b = 20;
console.log(a); // 10
console.log(b); // 20

// Reference (same object)
let obj1 = { name: "Abhi" };
let obj2 = obj1;
obj2.name = "TechHub";
console.log(obj1.name); // TechHub
```

Key Takeaway:

- Primitive types are immutable (value is copied).
- Reference types are mutable (same memory reference).

4 Dynamic Typing

JavaScript is dynamically typed, which means the type of a variable is determined at runtime and can change.

Example:

```
let value = 10; // number
console.log(typeof value); // "number"

value = "Hello"; // now it's a string
console.log(typeof value); // "string"

value = true; // now it's a boolean
console.log(typeof value); // "boolean"
```

Key Points:

- We don't need to declare the type of a variable explicitly.
- The type can change at runtime.
- Use typeof to check the current type.
- Makes JavaScript flexible but can also lead to unexpected bugs.

Pro Tip:

Use meaningful variable names and be careful while re-assigning values of different types.

JAVASCRIPT DEEP DIVE

50 Concepts You Must Know

"Learn
Build
Grow"

Page 2 : Execution & Scope

5 Execution Context

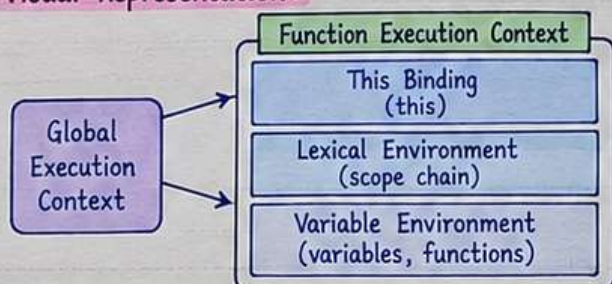
- Execution context is the environment in which JavaScript code is executed.
- Every time a function is called, a new execution context is created.
- It has three main components:
 - ① Variable Environment (stores variables)
 - ② Lexical Environment (handles scope chain)
 - ③ This Binding (value of this)

Example:

```
var x = 10;
function foo() {
  var y = 20;
  console.log(x + y);
}
foo(); // 30
```

When `foo()` is called, a new execution context is created with its own variable environment and lexical environment.

Visual Representation:



6 Global vs Function Scope

- Variables declared outside any function have global scope. They can be accessed anywhere in the code.
- Variables declared inside a function have function scope. They are only accessible within that function.

Example:

```
var a = 1; // global scope

function test() {
  var b = 2; // function scope
  console.log(a); // 1 (accessible)
  console.log(b); // 2 (accessible)
}

test();
console.log(a); // 1 (accessible)
// console.log(b); // Error: b is not defined
```

Global variables can be risky. Use them only when necessary.

Key Points:

- Global variables are added to the window object (in browser).
- Function scoped variables are not accessible outside the function.
- Always prefer block scope (`let/const`) over `var` to avoid scope-related issues.

7 Block Scope

- Variables declared with `let` and `const` are block scoped, meaning they are only accessible within the block (`{ }`) in which they are defined.
- `var` is not block scoped (it is function scoped).

Example:

```
{
  let x = 10;
  const y = 20;
  var z = 30;
  console.log(x); // 10
  console.log(y); // 20
  console.log(z); // 30
}
```

Block Scope Examples:

- if blocks
- for loops
- while loops
- try..catch
- `{ }` (any block)

```
// console.log(x); // Error: x is not defined
// console.log(y); // Error: y is not defined
console.log(z); // 30 (accessible, since var is function scoped)
```

Key Takeaway:

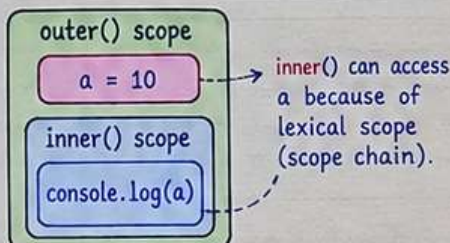
- Use `let` and `const` to avoid accidental global variables.
- Block scope helps in writing cleaner and more predictable code.

8 Lexical Scope

- Lexical scope means that a function can access variables from its outer (parent) scopes, even after the outer function has finished executing.
- The scope is determined at the time of writing the code (lexical), not at runtime.

Example:

```
function outer() {
  let a = 10;
  function inner() {
    console.log(a); // 10 (accesses from outer scope)
  }
  return inner;
}
const fn = outer();
fn(); // 10
```



Key Points:

- Inner functions have access to outer variables.
- Forms the basis of closures.

JAVASCRIPT DEEP DIVE

50 Concepts You Must Know

"Better Understanding Brings Fewer Bugs"

Page 3 : Hoisting & Temporal Dead Zone

9 Hoisting

- Hoisting is JavaScript's behavior of moving declarations to the top of their scope during the compilation phase (before code execution).
- Only declarations are hoisted, not initializations.
- var variables are hoisted and initialized with undefined.
- let and const are hoisted but kept in Temporal Dead Zone (not initialized).

Example:

```
console.log(a); // undefined
var a = 10;

console.log(b); // ReferenceError
let b = 10;

console.log(c); // ReferenceError
const c = 10;
```

Key Points:

- Declarations are hoisted.
- Initializations are not hoisted.
- var → undefined
- let/const → ReferenceError (in TDZ)

10 Temporal Dead Zone (TDZ)

- The time between entering a block and the actual declaration (initialization) of a let or const variable is called the Temporal Dead Zone.
- During this period, accessing the variable throws a ReferenceError.

Example:

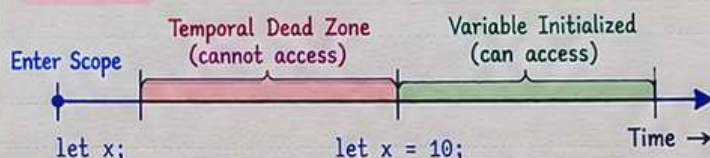
```
console.log(x); // ReferenceError
let x = 5;

if (true) {
  console.log(y); // ReferenceError
  const y = 10;
}
```

Why TDZ?

- Helps catch errors early.
- Prevents using variables before they are initialized.
- Makes code more predictable and less buggy.

Timeline:



11 Variable Creation Phase

- When JavaScript code runs, it goes through two main phases:
 - Creation Phase (Memory allocation)
 - Execution Phase (Code execution)
- During the creation phase:
 - Variables are hoisted and set to undefined (for var).
 - let and const are hoisted but not initialized (TDZ).
 - Function declarations are fully hoisted (with body).

Example:

Code	Creation Phase (Memory)
console.log(a); var a = 10; let b = 20; const c = 30; function foo() {}	var a → undefined let b → <uninitialized> (TDZ) const c → <uninitialized> (TDZ) function foo → entire function body

Execution Phase

- The code runs line by line.
- Variables get their actual values.
- Functions can be called.

Key Takeaway:

- Hoisting happens in the creation phase, before code execution.

12 Function Hoisting

- Function declarations are fully hoisted (both name and body).
- Function expressions are not hoisted (only the variable declaration is hoisted).
- Arrow functions (being expressions) are also not hoisted.

Example:

Function Declaration (hoisted)	Function Expression (not hoisted)
console.log(sum(2, 3)); // 5 function sum(a, b) { return a + b; }	console.log(mul(2, 3)); // TypeError: mul is not a function. var mul = function(a, b) { return a * b; };

Quick Comparison:

Type	Hoisted?	Can be used before declaration?	Value during creation phase
var variable	Yes	Yes (undefined)	undefined
let / const	Yes (in TDZ)	No (ReferenceError)	<uninitialized>
Function Declaration	Yes (fully)	Yes	Function body
Function Expression	Only variable	No (TypeError)	undefined
Arrow Function	Only variable	No (TypeError)	undefined

Pro Tip:

- Always declare functions before using them or use function declarations when hoisting behavior is needed.

JAVASCRIPT DEEP DIVE

50 Concepts You Must Know

"Functions
Make Code
Reusable,
Readable and
Powerful"

Page 4 : Functions Deep Dive

13 Function Declaration vs Expression

- Function declaration uses the function keyword and has a name (hoisted).
- Function expression assigns a function to a variable (not fully hoisted).
- Both create functions, but their hoisting behavior is different.

Function Declaration	Function Expression
<pre>function greet() { console.log("Hello"); }</pre> - Hoisted (can be called before definition) - Created during compilation phase - Has a name (recommended for reusability)	<pre>const greet = function() { console.log("Hello"); };</pre> - Not fully hoisted (variable is hoisted but initialized as undefined) - Created at runtime - Can be anonymous

Example:

```
// Works (due to hoisting)
sayHi();
function sayHi() {
  console.log("Hi");
}

// Error (not hoisted)
sayHello(); // TypeError
const sayHello = function() {
  console.log("Hello");
};
```

Key Takeaway:

- Use function declarations when hoisting is needed, otherwise prefer function expressions (more flexible).

14 Arrow Functions

- Arrow functions provide a shorter syntax for writing functions.
- They do not have their own this, arguments, super or new.target.
- They are best suited for small, concise functions.

Syntax:

```
// Traditional function
const add = function(a, b) {
  return a + b;
};

// Arrow function
const add = (a, b) => {
  return a + b;
};

// Implicit return (single line)
const sum = (a, b) => a + b;
```

Key Differences:

- No own this (inherits from surrounding scope)
- No arguments object
- Cannot be used as constructor (no new)
- Shorter syntax
- Best for callback functions

Example: this behavior

```
const obj = {
  name: "Abhi",
  regular: function() {
    console.log(this.name); // Abhi
  },
  arrow: () => {
    console.log(this.name); // undefined
  };
};
```

Use Cases:

- Array methods (map, filter, reduce)
- Callbacks (setTimeout, event listeners)
- When you don't need your own this

15 First-Class Functions

- In JavaScript, functions are first-class citizens (first-class functions), which means they can be treated like any other value.
- Functions can be:
 - Assigned to variables
 - Passed as arguments to other functions
 - Returned from other functions
 - Stored in data structures (arrays, objects)

Example:

```
// 1. Assign to a variable
const multiply = function(a, b) {
  return a * b;
};
console.log(multiply(2, 3)); // 6

// 2. Pass as argument
function operate(a, b, fn) {
  return fn(a, b);
}
console.log(operate(4, 5, multiply)); // 20

// 3. Return from a function
function createMultiplier(factor) {
  return function(num) {
    return num * factor;
  };
}
const double = createMultiplier(2);
console.log(double(5)); // 10
```

"Functions
are values,
just like numbers
or strings!"

Real World Use Cases:

- Event handlers
- Array methods
- Returning functions from factories
- Middleware (e.g., Express.js)

16 Higher-Order Functions

- A higher-order function is a function that either:
 - Takes one or more functions as arguments, or
 - Returns a function as its result.
- They are possible because JavaScript has first-class functions.

Example 1: Function as an argument

```
function greet(name, fn) {
  console.log("Hello " + name);
  fn();
}
greet("Abhi", () => console.log("Welcome!"));
```

map, filter,
reduce, forEach
are all
higher-order
functions!

Example 2: Function returning a function

```
function counter() {
  let count = 0;
  return function() {
    count++;
    return count;
  };
}

const increment = counter();
console.log(increment()); // 1
console.log(increment()); // 2
```

Common Higher-Order Functions in JS:

- map()
- filter()
- reduce()
- forEach()
- sort()
- setTimeout()
- event listeners
- custom HOFs

Key Takeaway:

- Higher-order functions make code more modular, reusable and expressive.
- They are widely used in modern JavaScript (especially in functional programming).

JAVASCRIPT DEEP DIVE

50 Concepts You Must Know

"Closures
Keep Variables
Alive"

Page 5 : Closures

17 Closures

- A closure is a function bundled together with its lexical environment (references to variables from its outer scope), even after the outer function has finished executing.
- In simple words, a closure allows a function to remember and access variables from its outer scope, even when that outer function has returned.

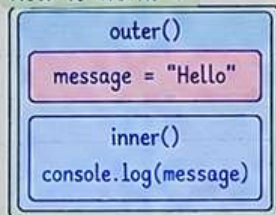
Closure = Function + Its Lexical Environment

Example :

```
function outer() {
  let message = "Hello";
  return function inner() {
    console.log(message); // Hello
  };
}

const fn = outer();
fn(); // Hello
```

How it works ?



Even after outer() returns, inner() still has access to message (closure).

Key Points :

- Closures are created at function creation time.
- They keep a reference to the outer scope.
- Outer variables are not copied, just referenced.
- Helps in data privacy and function factories.

19 Closures in Loops

- A common interview question where closures behave differently with var and let.
- When using var, all functions share the same variable (because var is function scoped).
- Using let creates a new binding for each iteration (block scoped), so each function gets its own copy.

Example 1 : Using var (Problem)

```
var funcs = [];
for (var i = 0; i < 3; i++) {
  funcs.push(function() {
    console.log(i);
  });
}

funcs[0](); // 3
funcs[1](); // 3
funcs[2](); // 3
```

All functions print 3 because they share the same variable i, and by the time they run, i = 3.

Example 2 : Using let (Correct)

```
var funcs = [];
for (let i = 0; i < 3; i++) {
  funcs.push(function() {
    console.log(i);
  });
}

funcs[0](); // 0
funcs[1](); // 1
funcs[2](); // 2
```

Each iteration has a new binding of i (0, 1, 2), so each function remembers its own value.



We can also use an IIFE with var to achieve the same result.

18 Closure + Private State

- Closures can be used to create private variables, which cannot be accessed directly from outside.
- This helps in data encapsulation (like private variables in classes).
- Each instance has its own separate lexical environment.

Example : Counter using Closure

```
function createCounter() {
  let count = 0; // private variable
  return {
    increment: function() {
      count++;
      console.log(count);
    },
    decrement: function() {
      count--;
      console.log(count);
    },
    getCount: function() {
      return count;
    }
  };
}
```

count is not accessible directly from outside!

console.log(count);
// ReferenceError

Output :

1
2
2

```
const counter = createCounter();
counter.increment(); // 1
counter.increment(); // 2
console.log(counter.getCount()); // 2
```

Key Takeaway :

- Closures enable data privacy.
- Used to create function-based modules.
- Useful before ES6 classes (and still relevant).

20 Practical Closure Use Cases

1 Function Factories

Closures allow us to create specialized functions with predefined data.

```
function multiply(factor) {
  return function(num) {
    return num * factor;
  };
}

const double = multiply(2);
console.log(double(5)); // 10
```

multiply(2) returns a function that remembers factor = 2

2 Memoization (Caching)

Closures can store results to avoid expensive computations.

```
function memoize(fn) {
  const cache = {};
  return function(n) {
    if (n in cache) return cache[n];
    cache[n] = fn(n);
    return cache[n];
  };
}
```

cache is private and persists across function calls.

3 Event Handlers

Closures are commonly used in event listeners to maintain state.

4 Module Pattern

Used to create private variables and expose only necessary methods (before ES6 modules).

Key Takeaway :

- Closures are powerful and widely used in real-world JavaScript.
- They help in data privacy, modularity, and performance optimization.

JAVASCRIPT DEEP DIVE

50 Concepts You Must Know

"Understand this, Avoid Confusion"

Page 6 : this Keyword Deep Dive

21 this Keyword

- this refers to the object that is executing the current function.
- The value of this is determined by HOW the function is called, not where it is defined.
- Its value changes based on the context (global, object, function, arrow, etc.).

Different contexts:

Context	Value of this	Example
Global context	window (browser) undefined (strict mode)	console.log(this);
Object method	the object itself	obj.method()
Regular function	window / undefined (in strict mode)	function() {}
Constructor function	newly created object	new Person()
Arrow function	inherits from parent scope	() => {}

Example:

```
console.log(this); // window (in browser)
function show() {
  console.log(this); // undefined (strict mode)
}
show();
```

Arrow functions do not have their own this. They inherit it from the enclosing scope.

24 bind() Method

- bind() does not invoke the function immediately.
- It returns a new function with the specified this value permanently bound.
- Syntax:** func.bind(thisArg, arg1, arg2, ...)

Example:

```
function greet(greeting) {
  console.log(greeting + " " + this.name);
}
const person = { name: "Abhi" };
const boundGreet = greet.bind(person, "Hello");
boundGreet(); // Hello Abhi
```

bind() is commonly used in event handlers and callbacks to preserve the context.

Real World Use Case:

```
const button = {
  name: "ClickMe",
  handleClick: function() {
    console.log("Button:", this.name);
  }
};
setTimeout(button.handleClick.bind(button), 1000);
// Button: ClickMe
```

Comparison: call vs apply vs bind

Method	Invokes Immediately?	Arguments	Returns
call()	Yes	Individual arguments	Function result
apply()	Yes	Array of arguments	Function result
bind()	No (returns new function)	Individual arguments	New function

22 call() Method

- call() invokes a function immediately with a specified this value and individual arguments.
- Syntax:** func.call(thisArg, arg1, arg2, ...)

Example:

```
function greet(greeting) {
  console.log(greeting + " " + this.name);
}
const person = { name: "Abhi" };
greet.call(person, "Hello"); // Hello Abhi
```

call() is useful when you want to invoke a function immediately and specify this manually.

23 apply() Method

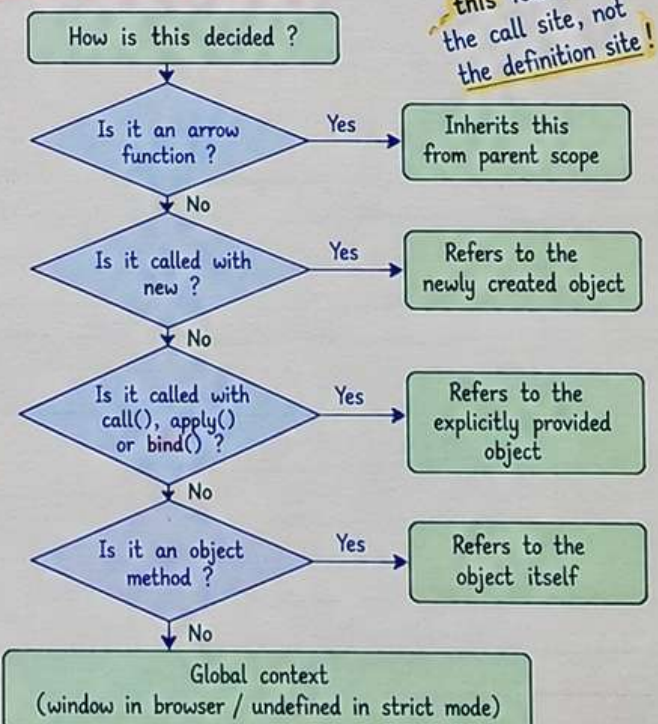
- apply() is similar to call(), but it takes arguments as an array (or array-like object).
- Syntax:** func.apply(thisArg, [arg1, arg2, ...])

Example:

```
function greet(greeting, age) {
  console.log(greeting + ", I am " + this.name + " and I am " + age);
}
const person = { name: "Abhi" };
greet.apply(person, ["Hi", 25]); // Hi, I am Abhi and I am 25
```

apply() is useful when you already have the arguments in an array.

How is this Decided?



Key Takeaways:

- this is determined by how the function is called.
- Use call() or apply() to invoke immediately with a specific context.
- Use bind() when you want a new function with a permanent context.
- Arrow functions inherit this from their parent scope.

JAVASCRIPT DEEP DIVE

50 Concepts You Must Know

"Objects Connect Everything in JavaScript"

Page 7 : Objects & Prototypes

25 Objects

- Objects are collection of key-value pairs.
- Keys are strings (or Symbols) and values can be of any type.
- Objects are reference types (stored in heap).
- There are multiple ways to create objects.

Examples :

```
// 1. Object literal (most common)
const user = {
  name: "Abhi",
  age: 25
};
console.log(user.name); // Abhi
console.log(user["age"]); // 25
```

```
// 2. Using new Object()
const obj = new Object();
obj.key = "value";
```

```
// 3. Using Object.create()
const obj2 = Object.create(null);
obj2.name = "Test";
```

Property Access :

- Dot notation → obj.key
- Bracket notation → obj["key"]
- Use bracket notation when key has spaces or is dynamic.

Key Points :

- Objects are mutable (can be changed).
- They are stored by reference.
- Comparing objects checks reference, not value.
- Use spread ({...obj}) for shallow copy.

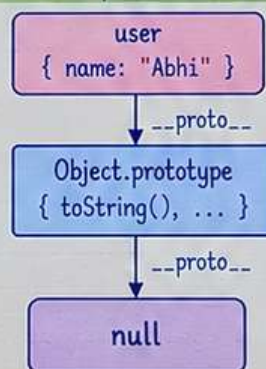
26 Prototype Chain

- Every JavaScript object has an internal link to another object called its prototype.
- When we try to access a property, JS first looks in the object itself, then in its prototype, and so on (prototype chain).
- At the end of the chain is Object.prototype, which links to null.

Example :

```
const user = { name: "Abhi" };
console.log(user.toString()); // from Object.prototype
```

Visual Representation :



Property Lookup :

- ① Check in the object itself.
- ② If not found, check in its prototype.
- ③ Repeat until found or reached null.

Example :

```
user.toString()
• Not found in user
• Found in Object.prototype
• So it works!
```

27 __proto__ vs prototype

- Both are related to prototypes but used in different ways.
- **prototype** is a property of constructor functions.
- **__proto__** is the internal prototype of an object (links to its parent).

__proto__	prototype
Exists on all objects (except null)	Exists only on constructor functions
Points to the prototype of the object	Used to set the prototype for objects created by the constructor
Used for inheritance (property lookup)	Used when creating objects with new
Accessed as obj.__proto__	Accessed as Constructor.prototype
Not recommended to use directly (use Object.getPrototypeOf)	Commonly used and safe

Example :

```
function Person(name) {
  this.name = name;
}
const p = new Person("Abhi");

console.log(p.__proto__ === Person.prototype); // true
console.log(Person.prototype); // { constructor: f }
```

Interview Tip :

- **__proto__** is about inheritance (chain).
- **prototype** is about object creation with new.

Key Takeaways :

- Everything in JavaScript is an object (except primitives).
- Prototypes enable inheritance and code reusability.
- Use constructor functions (or classes) when you need to create multiple similar objects.

28 Constructor Functions

- Constructor functions are regular functions used with the new keyword to create objects.
- Inside a constructor, **this** refers to the newly created object.
- The new keyword performs 4 steps behind the scenes.

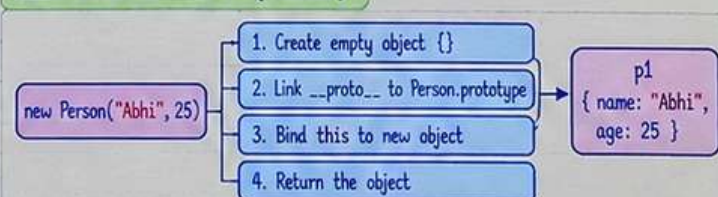
Example :

```
function Person(name, age) {
  this.name = name;
  this.age = age;
}
Person.prototype.greet = function() {
  console.log("Hi, I'm " + this.name);
};
const p1 = new Person("Abhi", 25);
p1.greet(); // Hi, I'm Abhi
```

4 Steps of new Keyword :

- ① Creates a new empty object.
- ② Sets the object's **__proto__** to Constructor.prototype.
- ③ Binds **this** to the new object.
- ④ Returns the new object (implicitly).

How new Works ? (Visual)



Pro Tip :

- In modern JavaScript, prefer ES6 classes (syntactic sugar over constructor functions) for better readability.
- Understanding prototypes is crucial for interviews!

JAVASCRIPT DEEP DIVE

50 Concepts You Must Know

"Classes
Make Inheritance
Simple
(But Know What
Happens Behind)"

Page 8 : Classes & Inheritance

29 ES6 Classes

- Introduced in ES6 (2015) as a cleaner syntax over prototype-based inheritance.
- Classes are syntactic sugar over constructor functions and prototypes.
- They do not introduce a new object-oriented model, but make it easier to write and understand.

Syntax :

```
class User {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }

  greet() {
    console.log('Hi, I'm ${this.name}');
  }
}

const u1 = new User("Abhi", 25);
u1.greet(); // Hi, I'm Abhi
```

Key Points :

- Use class keyword.
- constructor() is optional.
- Methods are added to the prototype (not recreated for every instance).
- Classes support inheritance using extends.

Behind the Scenes (Equivalent):

```
function User(name, age) {
  this.name = name;
  this.age = age;
}

User.prototype.greet = function() {
  console.log('Hi, I'm ${this.name}');
};
```

Classes are just a cleaner way to write prototype-based code !

30 Constructor

- The constructor() method is a special method for creating and initializing an object created with a class.
- It is called automatically when a new instance is created using the new keyword.
- We use this to set initial values (instance properties).

Example :

```
class Product {
  constructor(name, price) {
    this.name = name; // instance property
    this.price = price; // instance property
  }

  show() {
    console.log(`${this.name} - ₹${this.price}`);
  }
}

const p = new Product("Laptop", 75000);
p.show(); // Laptop - ₹75000
```

Important :

- constructor() is called with new keyword.
- this refers to the newly created object.
- If we don't define constructor(), JavaScript provides a default empty constructor.

Instance Properties vs Prototype Methods

Instance Properties (inside constructor)	Prototype Methods (class methods)
- Added to each object (instance). - Each instance gets its own copy. - Use for unique data (name, age, etc).	- Added to the prototype. - Shared across all instances. - More memory efficient. - Use for common behavior (greet()).

31 extends (Inheritance)

- The extends keyword is used to create a child class that inherits properties and methods from a parent class.
- The child class gets access to the parent class's prototype chain.
- We can also override parent methods in the child class.

Example :

```
class Person {
  constructor(name) {
    this.name = name;
  }

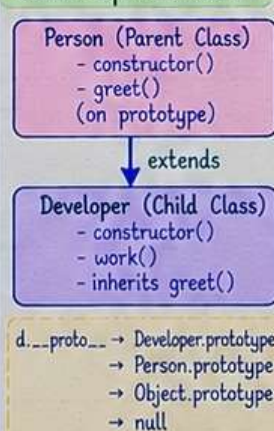
  greet() {
    console.log('Hi, I'm ${this.name}');
  }
}

class Developer extends Person {
  constructor(name, role) {
    super(name); // call parent constructor
    this.role = role;
  }

  work() {
    console.log(`${this.name} is a ${this.role}`);
  }
}

const d = new Developer("Abhi", "Frontend Dev");
d.greet(); // Hi, I'm Abhi
d.work(); // Abhi is a Frontend Dev
```

Visual Representation :



Check Inheritance :

```
console.log(d instanceof Developer); // true
console.log(d instanceof Person); // true
```

32 super Keyword

- The super keyword is used to call the parent class's constructor or methods.
- It must be called before using this in the child class constructor.
- It can also be used to call parent class methods.

Example 1 : Calling parent constructor

```
class Car {
  constructor(brand) {
    this.brand = brand;
  }
}

class ElectricCar extends Car {
  constructor(brand, battery) {
    super(brand); // calls Car constructor
    this.battery = battery;
  }
}

const tesla = new ElectricCar("Tesla", "75kWh");
console.log(tesla.brand); // Tesla
console.log(tesla.battery); // 75kWh
```

Example 2 : Calling parent method

```
class Animal {
  speak() {
    console.log("Animal makes a sound");
  }
}

class Dog extends Animal {
  speak() {
    super.speak(); // call parent method
    console.log("Dog barks");
  }
}

const dog = new Dog();
dog.speak();
// Animal makes a sound
// Dog barks
```

Rules to Remember :

- super() must be called before using this in child constructor.
- super can only be used in a derived class (class extending another class).
- It can be used to call parent methods using super.method().
- It cannot be used outside classes.

Common Use Cases :

- Call parent constructor.
- Call parent methods (when overriding).
- Useful in multi-level inheritance.

Key Takeaway :

- extends and super make inheritance easy in JS.
- Understand prototype chain behind it for interviews.
- Classes are just syntactic sugar over prototypes!

JAVASCRIPT DEEP DIVE

50 Concepts You Must Know

"Modern JavaScript Makes You Write Less Code and Do More"

Page 9 : Arrays & Modern JavaScript

33 Destructuring

- Destructuring allows us to extract values from arrays or properties from objects into variables.
- It provides a clean and concise syntax.
- We can also set default values and use nested destructuring.

Array Destructuring :

```
const numbers = [10, 20, 30];
const [a, b, c] = numbers;
console.log(a, b, c); // 10 20 30
```

Order matters in array destructuring!

Object Destructuring :

```
const user = { name: "Abhi", age: 25 };
const { name, age } = user;
console.log(name, age); // Abhi 25
```

Default Values :

```
const [x, y = 5] = [1];
console.log(x, y); // 1 5
```

Nested Destructuring :

```
const data = { user: { name: "Abhi" } };
const { user: { name } } = data;
console.log(name); // Abhi
```

Key Takeaways :

- Destructuring makes code cleaner and more readable.
- Useful when working with function returns (e.g., useState, API responses).

34 Spread vs Rest

- Both spread and rest use the same syntax (...), but they work differently.
- Spread expands an iterable into individual elements.
- Rest collects multiple elements into an array.

Spread Operator (...)

- Expands elements (arrays, objects).
- Used to copy, merge, or pass values.
- Commonly used in array/object literals and function calls.

Example :

```
const arr1 = [1, 2, 3];
const arr2 = [...arr1, 4, 5];
console.log(arr2); // [1, 2, 3, 4, 5]
```

```
const obj1 = { a: 1 };
const obj2 = { ...obj1, b: 2 };
console.log(obj2); // { a: 1, b: 2 }
```

Rest Parameter (...)

- Collects multiple elements into a single array.
- Used in function parameters and destructuring.

Example :

```
function sum(...nums) {
  return nums.reduce((a, b) => a + b, 0);
}
console.log(sum(1, 2, 3)); // 6
```

```
const [first, ...rest] = [10, 20, 30, 40];
console.log(first); // 10
console.log(rest); // [20, 30, 40]
```

Spread vs Rest :

Feature	Spread (...)	Rest (...)
Purpose	Expands elements	Collects elements
Usage	In array/object literals, function calls	In function parameters, destructuring
Example	[...arr], {...obj}	function fn(...args), const [...rest] = arr
Returns	Individual elements	Array of elements

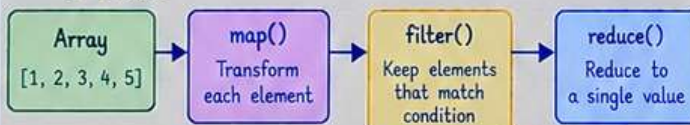
Common Trap :

... in function parameters is rest, not spread!

```
function fn(...args) {}
// rest, not spread
```

35 map() / filter() / reduce()

- These are higher-order array methods used to transform, filter, and reduce data.
- They do not mutate the original array (return a new array or value).



map()

- Creates a new array by transforming each element.

Example :

```
const nums = [1, 2, 3];
const squares = nums.map(
  n => n * n
);
console.log(squares);
// [1, 4, 9]
```

filter()

- Creates a new array with elements that pass a condition.

Example :

```
const nums = [1, 2, 3, 4, 5];
const even = nums.filter(
  n => n % 2 === 0
);
console.log(even);
// [2, 4]
```

reduce()

- Reduces the array to a single value (accumulator).

Example :

```
const nums = [1, 2, 3, 4];
const sum = nums.reduce(
  (acc, curr) => acc + curr,
  0
);
console.log(sum);
// 10
```

When to Use ?

- map() → When you want to transform each element.
- filter() → When you want to keep only certain elements.
- reduce() → When you want to combine all elements into a single value.

36 Optional Chaining & Nullish Coalescing

- Optional chaining (?.) allows safe access to nested properties without throwing an error if a value is null or undefined.
- Nullish coalescing (??) provides a default value when the left-hand side is null or undefined (but not for other falsy values like 0, "", or false).

Optional Chaining (?.)

- If the value before ?. is null or undefined, it returns undefined instead of throwing an error.

Example :

```
const user = {
  name: "Abhi",
  address: {
    city: "Delhi"
  }
};
console.log(user?.address?.city);
// Delhi
console.log(user?.phone?.number);
// undefined (no error)
```

Nullish Coalescing (??)

- Returns the right-hand side value only if the left-hand side is null or undefined.

Example :

```
const name = null;
const displayName = name ?? "Guest";
console.log(displayName);
// Guest

const count = 0;
const value = count ?? 10;
console.log(value);
// 0 (not 10)
```

?? vs ||

- || returns the right side for all falsy values (0, "", false, null, undefined).
- ?? returns the right side only for null or undefined.

Value	value "X"	value ?? "X"
0	"X"	0
""	"X"	""
false	"X"	false
null	"X"	"X"
undefined	"X"	"X"

Key Takeaways :

- Use optional chaining to avoid TypeErrors.
- Use nullish coalescing when you want a default value only for null/undefined.
- These features make your code safer and cleaner.

JAVASCRIPT DEEP DIVE

50 Concepts You Must Know

"Async JS
Looks Complicated,
But It's Just
Event Loop Magic"

Page 10 : Event Loop & Asynchronous JavaScript

37 Call Stack

- The call stack is a data structure that keeps track of function calls (LIFO - Last In First Out).
- When a function is invoked, it is pushed onto the stack and when it finishes, it is popped off.
- If the stack size exceeds the limit, we get a "Stack Overflow" error.

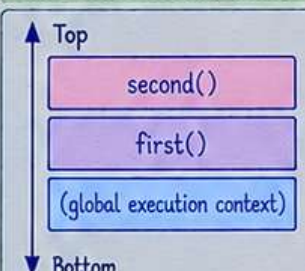
Example :

```
function first() {
  console.log("First");
  second();
}

function second() {
  console.log("Second");
}

first();
```

Call Stack Visualization :



Key Points :

- Synchronous code is executed in the call stack.
- Only one thing runs at a time in the call stack.
- A function stays in the stack until it completes.
- Stack overflow happens with too much recursion.

38 Web APIs / Runtime APIs

- Web APIs are provided by the browser (not part of JavaScript engine).
- They handle time-consuming tasks like timers, DOM events, network requests (fetch), etc.
- After the task is completed, its callback is pushed to the task queue (macrotask queue).

Examples of Web APIs :

- setTimeout() / setInterval()
- DOM events (click, scroll, etc.)
- fetch() (network requests)
- LocalStorage / sessionStorage
- Canvas, Geolocation, etc.

Important :

Web APIs run outside the JavaScript engine. They don't block the call stack.

Example : setTimeout

```
console.log("Start");
setTimeout(() => {
  console.log("After 2 seconds");
}, 2000);
console.log("End");
```

Output :

Start
End
After 2 seconds
(because setTimeout is handled by Web APIs)

39 Task (Macrotask) Queue

- Also called the callback queue.
- Holds callbacks from Web APIs like setTimeout, DOM events, I/O, etc.
- Callbacks are moved to the call stack only when it is empty.
- Executed one by one (FIFO).

Examples :

- setTimeout()
- setInterval()
- DOM events (click, scroll, etc.)
- I/O callbacks
- setImmediate() (in Node.js)

Note :

Even if setTimeout has a delay of 0, its callback goes to the task queue and will run after all microtasks are completed.

40 Microtask Queue

- Holds high-priority tasks like Promise callbacks, queueMicrotask().
- Microtasks run before macrotasks (task queue).
- After the current script finishes, the event loop first executes all microtasks, and then moves to the next macrotask.

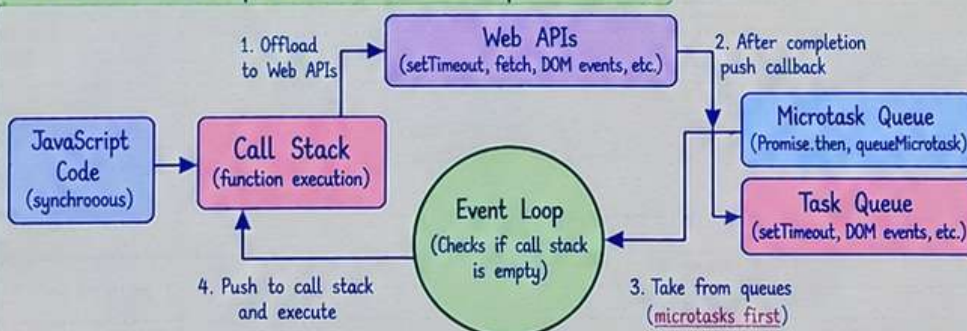
Examples :

- Promise.then() / .catch() / .finally()
- queueMicrotask()
- MutationObserver (in browsers)
- process.nextTick() (in Node.js, even higher priority than other microtasks)

Important :

Microtasks always run before macrotasks. If a microtask adds another microtask, it will also run before moving to the task queue.

How the Event Loop Works ? (Visual Representation)



What is the Output ?

```
console.log("1");
setTimeout(() => {
  console.log("2");
}, 0);
Promise.resolve().then(() => {
  console.log("3");
});
console.log("4");
```

Output :

1
4
3
2

Explanation :

- console.log("1") → sync
- console.log("4") → sync
- Promise.then() → microtask
- setTimeout() → macrotask

Key Takeaways :

- JavaScript is single-threaded but non-blocking.
- The event loop enables asynchronous behavior.
- Microtasks (Promises) run before macrotasks (setTimeout).
- Understanding the event loop is crucial for interviews and writing optimized code.

"The event loop isn't magic,
it's just a queue with rules."

JAVASCRIPT DEEP DIVE

50 Concepts You Must Know

"From Callbacks to Async/Await JavaScript Keeps Evolving"

Page 11 : Async JavaScript & Promises

41 Callbacks

- A callback is a function passed as an argument to another function, which is executed later.
- Callbacks can be synchronous or asynchronous.
- Widely used for handling async operations before Promises (e.g., `setTimeout`, event listeners, file I/O).

Example :

```
function fetchData(callback) {
  setTimeout(() => {
    console.log("Data fetched");
    callback();
  }, 1000);
}

fetchData(() => {
  console.log("Callback executed");
});
// Data fetched (after 1s)
// Callback executed
```

Callback Hell :

- Multiple nested callbacks make code difficult to read and maintain.

```
setTimeout(() => {
  setTimeout(() => {
    setTimeout(() => {
      console.log("...");
    }, 1000);
  }, 1000);
}, 1000);
```

Error-First Callback Pattern :

- Common in Node.js
- First argument is error (if any), second is result.

```
fs.readFile("file.txt", (err, data) => {
  if (err) return console.log("Error:", err);
  console.log("Data:", data);
});
```

42 Promises

- A Promise is an object that represents the eventual completion (or failure) of an asynchronous operation.
- It can be in one of three states: **pending**, **fulfilled**, or **rejected**.
- Promises solve the problem of callback hell and make async code more readable.

Syntax :

```
const promise = new Promise((resolve, reject) => {
  // do some async task
  if (success) {
    resolve("Success");
  } else {
    reject("Error");
  }
});
```

Key Methods :

- `.then()` → handles success
- `.catch()` → handles error
- `.finally()` → runs always

Example :

```
const promise = new Promise((resolve, reject) => {
  setTimeout(() => resolve("Done"), 1000);
});

promise
  .then(result => console.log(result)) // Done
  .catch(err => console.log(err))
  .finally(() => console.log("Finally executed"));
```

Advantages :

- Avoids callback hell
- Better error handling
- Easier to chain operations
- More readable code

43 async / await

- `async/await` is syntactic sugar over Promises.
- An `async` function always returns a Promise.
- `await` pauses the execution of the `async` function until the Promise is resolved or rejected.
- Use `try/catch` to handle errors.

Example :

```
async function getData() {
  try {
    const result = await fetch("https://api.example.com");
    const data = await result.json();
    console.log(data);
  } catch (error) {
    console.log("Error:", error);
  }
}

getData();
```

Key Points :

- `async` always returns a Promise.
- `await` can only be used inside `async` functions.
- Helps write asynchronous code in a synchronous and cleaner way.

Sequential vs Parallel Execution :

Sequential (one by one)	Parallel (using <code>Promise.all</code>)
<pre>await task1(); await task2(); await task3();</pre>	<pre>await Promise.all([task1(), task2(), task3()]);</pre>
Total time = sum of all tasks	Total time = time of longest task

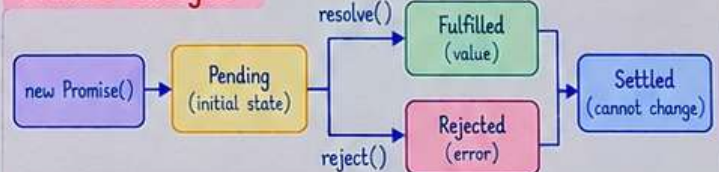
Key Takeaways :

- Callbacks → powerful but can lead to callback hell.
- Promises → solve callback hell and make async code manageable.
- `async/await` → makes asynchronous code look synchronous.
- Understanding async JS is crucial for real-world applications (APIs, databases, etc.).

44 Promise States & Lifecycle

- A Promise starts in the **pending** state.
- It can either be **fulfilled** (resolved) or **rejected**.
- Once settled (fulfilled or rejected), it cannot change its state.
- You can chain multiple `.then()` calls, and each `.then()` returns a new Promise.

Promise Lifecycle :



Chaining & Returned Values :

```
Promise.resolve(1)
  .then(val => val + 1) // 2
  .then(val => val * 2) // 4
  .then(val => console.log(val));
```

Important :

- Once a Promise is settled, its state cannot change.
- `.then()` always returns a new Promise.
- You can return a value or another Promise from `.then()`.

What is the Output ?

```
console.log("1");
setTimeout(() => console.log("2"), 0);
Promise.resolve().then(() => console.log("3"));
console.log("4");
```

Output :

1
4
3
2

Explanation :

1. `console.log("1")` → sync
2. `console.log("4")` → sync
3. `Promise.then()` → microtask
4. `setTimeout()` → macrotask

"Asynchronous code isn't hard, it just needs the right mental model."

JAVASCRIPT DEEP DIVE

50 Concepts You Must Know

"Different Problems, Different Promise Combinators"

Page 12 : Promise Combinators

45 Promise.all()

- Runs multiple promises concurrently.
- Resolves when all promises are fulfilled.
- If any promise rejects, the whole Promise rejects immediately (fails fast).
- Useful when you need all results (e.g., multiple APIs).

Example :

```
const p1 = Promise.resolve("A");
const p2 = Promise.resolve("B");
const p3 = Promise.resolve("C");

Promise.all([p1, p2, p3])
  .then(res => console.log(res));
// ["A", "B", "C"]
```

If any promise rejects, it immediately rejects and other results are ignored (even if they resolve later).

46 Promise.allSettled()

- Waits for all promises to settle (either fulfilled or rejected).
- Never rejects because of an individual failure.
- Returns an array of result objects with status and value/reason.
- Useful when you want to know the outcome of every promise (e.g., multiple independent tasks).

Example :

```
const p1 = Promise.resolve("A");
const p2 = Promise.reject("Error");
const p3 = Promise.resolve("C");

Promise.allSettled([p1, p2, p3])
  .then(res => console.log(res));
```

Output :

```
[
  { status: "fulfilled", value: "A" },
  { status: "rejected", reason: "Error" },
  { status: "fulfilled", value: "C" }
]
```

47 Promise.race()

- Returns the first settled promise (fulfilled or rejected).
- Does not wait for other promises.
- Useful for timeout patterns or when you need the fastest result.

Example :

```
const p1 = new Promise((res) =>
  setTimeout(() => res("First"), 1000));
const p2 = new Promise((res) =>
  setTimeout(() => res("Second"), 2000));

Promise.race([p1, p2])
  .then(res => console.log(res));
// "First" (after 1s)
```

The first promise to settle (resolve or reject) wins. Others are ignored (but still keep running in background).

48 Promise.any()

- Returns the first fulfilled promise.
- Ignores rejected promises (until all reject).
- If all promises reject, it rejects with an AggregateError.
- Useful when you need the first successful result (e.g., fallback APIs).

Example :

```
const p1 = Promise.reject("Error 1");
const p2 = new Promise((res) =>
  setTimeout(() => res("Success"), 1000));
const p3 = Promise.reject("Error 3");

Promise.any([p1, p2, p3])
  .then(res => console.log(res));
// "Success"
```

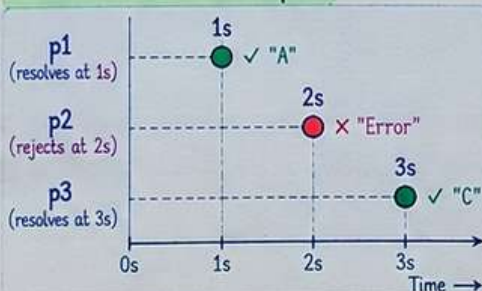
All Reject Example :

```
Promise.any([
  Promise.reject("E1"),
  Promise.reject("E2")
]).catch(err =>
  console.log(err));
// AggregateError:
// All promises were
// rejected
```

Comparison Table :

Feature	Promise.all()	Promise.allSettled()	Promise.race()	Promise.any()
What it does	Waits for all to fulfill	Waits for all to settle	Returns first settled	Returns first fulfilled
When it resolves	When all fulfill	When all settle	When first settles	When first fulfills
When it rejects	If any rejects (fails fast)	Never rejects (always resolves)	If first settles is a rejection	Only if all reject (AggregateError)
Result	Array of values	Array of { status, value/reason }	Single value or reason	Single fulfilled value
Use case	All results required (e.g., multiple APIs)	Need to know outcome of every promise	Timeout, fastest response	First successful result (fallbacks)

Visual Timeline Example



Results for the above timeline :

```
Promise.all([p1, p2, p3])
  → Rejects with "Error" (at 2s)

Promise.allSettled([p1, p2, p3])
  → [{fulfilled: "A"}, {rejected: "Error"}, {fulfilled: "C"}]

Promise.race([p1, p2, p3])
  → "A" (at 1s)

Promise.any([p1, p2, p3])
  → "A" (at 1s)
```

Interview Questions :

① What is the output ?

```
const p1 = Promise.resolve(1);
const p2 = Promise.reject(2);
const p3 = Promise.resolve(3);
Promise.all([p1, p2, p3])
  .then(console.log)
  .catch(console.log);
```

Output :
2

② What is the output ?

```
const p1 = new Promise((res) => setTimeout(() => res("A"), 1000));
const p2 = new Promise((res) => setTimeout(() => res("B"), 0));

console.log("Start");
Promise.race([p1, p2]).then(console.log);
console.log("End");
```

Output :
Start
End
B

Key Takeaways :

- Choose the right combinator based on the problem.
- all() → All must succeed.
- allSettled() → Get results of all.
- race() → First settled result.
- any() → First successful result.

"Promises give you options, Combinators give you power!"

JAVASCRIPT DEEP DIVE

50 Concepts You Must Know

"Good Developers Write Code.
Great Developers Also Manage Memory."

Page 13 : Memory Management & Memory Leaks

49 Garbage Collection

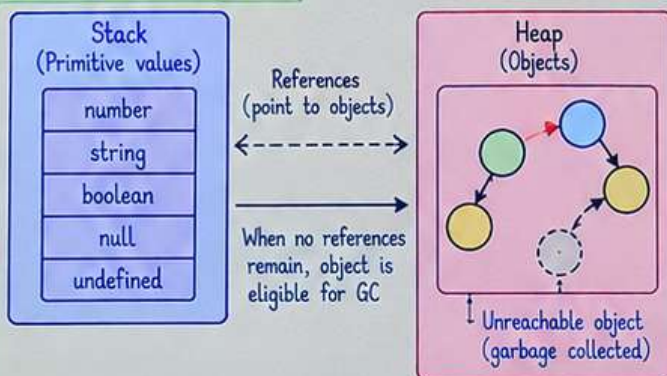
- JavaScript automatically manages memory using Garbage Collection (GC).
- GC frees up memory that is no longer reachable from the root (e.g., global object).
- Most modern JavaScript engines (like V8) use a Mark-and-Sweep algorithm.
- We cannot manually trigger GC (except in some environments like Chrome DevTools).

How Mark-and-Sweep Works ?

- Mark:** Start from root (global object) and mark all reachable objects.
- Sweep:** Remove all unmarked (unreachable) objects from memory.
- Reclaim:** Freed memory is made available for new allocations.

If an object is not reachable (from any execution context), it becomes eligible for garbage collection.

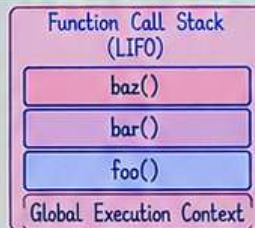
Visual Representation



50 Stack vs Heap

- Stack is used for primitive values and function calls (fast, fixed size).
- Heap is used for objects, arrays, functions (dynamic size).
- Memory in stack is automatically cleaned up when the function call completes (LIFO).
- Objects in heap are cleaned up by Garbage Collector when they are no longer reachable.

Call Stack (Stack Memory)



- Stores function calls and primitive values.
- Automatically cleared when function execution completes.

Heap Memory

Objects (stored in heap)



- Allocated dynamically.
- Garbage collected when no longer reachable.

Common Causes of Memory Leaks

- Accidental Global Variables**
Variables declared without let, const, or var become global and stay in memory.
- Forgotten Timers**
setInterval or setTimeout not cleared.
- Event Listeners**
Listeners not removed (especially on DOM elements).
- Detached DOM Nodes**
DOM nodes removed from the document but still referenced.
- Closures**
Functions retaining references to large data even when not needed.
- Caches & Large Data Structures**
Data kept in memory without proper cleanup.

Example 1 : Accidental Global Variable

Bad (Memory Leak)

```
function createUser(name) {
  user = { name, age: 25 }; // no let/const/var
}

createUser("Abhi");
console.log(user); // still accessible globally
```

Good (No Leak)

```
function createUser(name) {
  const user = { name, age: 25 };
  return user;
}

const u = createUser("Abhi");
console.log(u);
```

Example 2 : Event Listener Leak

Bad (Listener not removed)

```
function addButton() {
  const btn = document.createElement("button");
  btn.addEventListener("click", () => {
    console.log("Clicked");
  });
  document.body.appendChild(btn);
  // If btn is later removed but listener
  // is not cleaned, it can cause a leak
}
```

Good (Remove listener)

```
function addButton() {
  const btn = document.createElement("button");
  const handleClick = () => console.log("Clicked");
  btn.addEventListener("click", handleClick);
  // Later when removing
  btn.removeEventListener("click", handleClick);
  btn.remove();
}
```

WeakMap and WeakSet

- WeakMap and WeakSet hold weak references to objects.
- They do not prevent garbage collection.
- Useful for caching and storing metadata without causing memory leaks.

Example :

```
const wm = new WeakMap();
let obj = { name: "Abhi" };
wm.set(obj, "some data");
obj = null; // now object can be garbage collected
```

Use WeakMap/WeakSet when you don't want the stored objects to prevent garbage collection.

How to Prevent Memory Leaks ?

- Always use let/const/var to declare variables.
- Clear timers (clearInterval / clearTimeout).
- Remove event listeners when not needed.
- Avoid keeping unnecessary references in closures.
- Use WeakMap / WeakSet for caches.
- Set large object references to null when done.
- Use browser DevTools (Memory tab) to detect leaks.

Interview Question

Q: JavaScript is garbage-collected, so how can memory leaks still happen?

A: Because objects can still be reachable even if we no longer need them. If references to those objects are not removed (e.g., in closures, global variables, event listeners, timers), the GC cannot collect them, leading to memory leaks.

"Memory leaks don't crash your app immediately, but they slowly kill performance."

JAVASCRIPT DEEP DIVE

50 Concepts You Must Know

"Understand the Browser, Because That's Where Your JavaScript Comes Alive"

Page 14 : Browser JavaScript & DOM

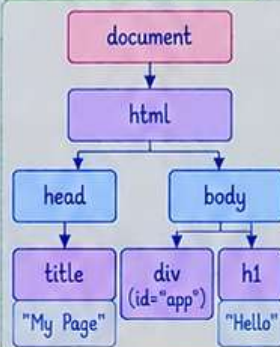
51 DOM & DOM Tree

- DOM (Document Object Model) is a tree representation of the HTML document.
- It allows JavaScript to access and manipulate HTML elements.
- Each HTML element is a node (element, attribute, text, etc.).
- Changes in the DOM are reflected in the UI.

Example :

```
const title = document.getElementById("title");
title.textContent = "Hello World!";
title.style.color = "blue";
```

DOM Tree (Simplified)



We can traverse, add, remove or modify nodes using JavaScript.

Code Example : Event Delegation

```
<ul id="list">
  <li>Item 1</li>
  <li>Item 2</li>
  <li>Item 3</li>
</ul>

const list = document.getElementById("list");
list.addEventListener("click", (e) => {
  if (e.target.tagName === "LI") {
    console.log("Clicked:"; e.target.textContent);
  }
}); //single listener for all <li> elements
```

Benefits :

- Better performance
- Less memory usage
- Works for dynamically added elements

52 Event Bubbling, Capturing & Delegation

- When an event occurs, it goes through three phases: capturing → target → bubbling.
- By default, event listeners run in the bubbling phase.
- Event delegation uses a single listener on a parent element to handle events for multiple child elements.

Event Propagation (Order)

1. Capturing Phase
(root → target)

2. Target Phase
(event on target element)

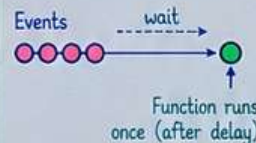
3. Bubbling Phase
(target → root)

Use event delegation when you have many similar elements (e.g., list items, buttons).

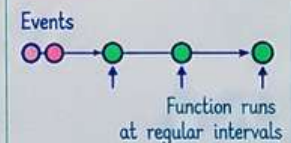
53 Debouncing vs Throttling

- Debouncing: Delays function execution until a certain time has passed after the last event.
- Throttling: Ensures the function runs at most once every specified time interval.

Debouncing
(wait for user to stop)



Throttling
(run at fixed intervals)



Use Cases :

- Debouncing → Search input, window resize (wait until user stops)
- Throttling → Scroll events, window resize, button clicks (limit execution rate)

54 LocalStorage vs SessionStorage vs Cookies

Feature	LocalStorage	SessionStorage	Cookies
Data Lifetime	Permanent (until manually cleared)	Until tab is closed	Custom (with expiry)
Size Limit	~ 5-10 MB	~ 5-10 MB	~ 4 KB
Sent to Server	No	No	Yes (with every request)
Accessible in JS	Yes	Yes	Yes
Use Case	Store user preferences, theme, etc.	Store temporary data for the session	Authentication, session management

LocalStorage Example :

```
localStorage.setItem("name", "Abhi");
console.log(localStorage.getItem("name"));
localStorage.removeItem("name");
```

SessionStorage Example :

```
sessionStorage.setItem("temp", "123");
console.log(sessionStorage.getItem("temp"));
```

Cookie Example :

```
document.cookie = "user=Abhi; expires=Fri, 31 Dec 2025 23:59:59 GMT; path=/";
console.log(document.cookie);
```

Key Takeaways :

- DOM allows JavaScript to interact with the webpage.
- Events go through capturing → target → bubbling.
- Use debouncing and throttling to optimize performance.
- Choose the right storage (localStorage, sessionStorage, cookies) based on your use case.

55 Other Useful Browser APIs

- DOM APIs: getElementById, querySelector, createElement, etc.
- Window APIs: setTimeout, setInterval, requestAnimationFrame, alert, confirm, prompt.
- Navigator API: information about browser (navigator.userAgent).
- Location API: window.location (URL handling).
- History API: back, forward, pushState (SPA routing).
- Fetch API: to make network requests.
- Web Storage API: localStorage, sessionStorage.
- Cookies: document.cookie.

Interview Question :

- Q: Why is event delegation better than adding listeners to hundreds of elements?
- A: It improves performance, reduces memory usage, and also works for dynamically added elements.

"The browser gives you the playground, JavaScript gives you the superpowers."

JAVASCRIPT DEEP DIVE

50 Concepts You Must Know

"Don't just write JavaScript. Understand what happens behind the code."

Page 15 : JavaScript Interview Master Sheet

Top 10 Most Asked JavaScript Questions

- 1 Explain the Event Loop**
 - Call Stack, Web APIs
 - Microtask vs Macrotask
 - Execution flow (Page 10)
- 2 What is a Closure?**
 - Function + lexical scope
 - Access to outer variables
 - Real world use cases (Page 5)
- 3 Explain Hoisting & TDZ**
 - var / let / const hoisting
 - Temporal Dead Zone
 - Why let/const are safer (Page 3)
- 4 How does "this" work?**
 - this in global, function, object, arrow, class
 - call, apply, bind (Page 6)
- 5 Explain the Prototype Chain**
 - __proto__ vs prototype
 - Prototype inheritance
 - Object.create(), new keyword (Page 7)
- 6 == vs ===**
 - Type coercion
 - Abstract vs Strict equality
 - Common gotchas (Page 4)
- 7 Promise vs async/await**
 - Syntax, error handling
 - When to use what
 - Real world example (Page 11)
- 8 call() vs apply() vs bind()**
 - How they work
 - Differences
 - Use cases (Page 6)
- 9 Microtask vs Macrotask**
 - Execution order
 - Promise vs setTimeout
 - Event loop flow (Page 10)
- 10 How do memory leaks happen?**
 - Common causes
 - How to prevent
 - Real world examples (Page 13)

Rapid-Fire Comparisons

var vs let vs const

- var : function scoped
- let : block scoped
- const : block scoped (reassign X)

map() vs filter() vs reduce()

- map() : transforms (same length)
- filter() : filters (<= length)
- reduce() : reduces to single value

Spread vs Rest

- Spread : expands (...arr)
- Rest : collects (...args)
- Used in different contexts

null vs undefined

- null : intentional absence
- undefined : not assigned
- typeof null → "object"

|| vs ??

- || : returns first truthy value
- ?? : returns first non-null/undefined
- Use ?? for default values

slice() vs splice()

- slice() : does not modify array
- splice() : modifies original array

Promise.all() vs allSettled()

- all() : all must succeed
- allSettled() : waits for all
- race() : first settled
- any() : first fulfilled

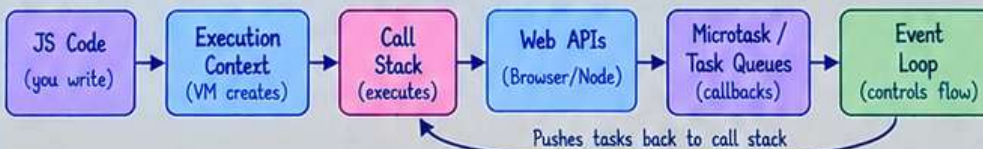
Debounce vs Throttle

- Debounce : run after delay (stops frequent calls)
- Throttle : run at fixed intervals (limits rate)

__proto__ vs prototype

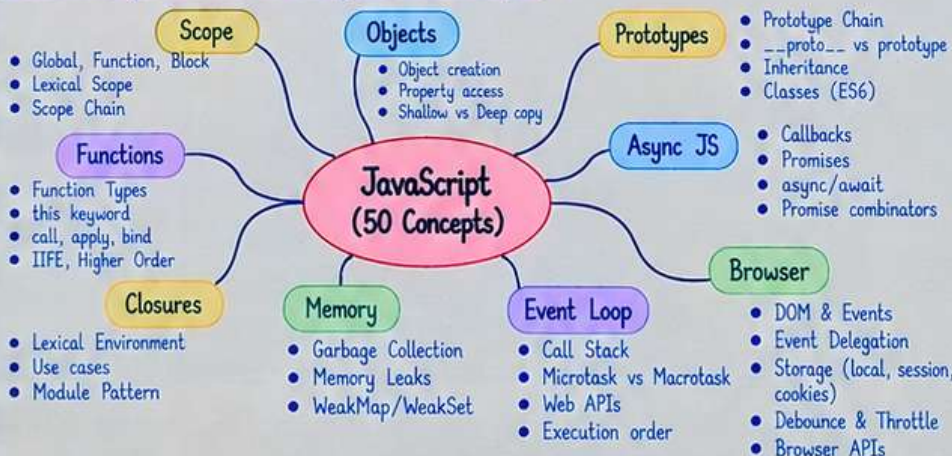
- __proto__ : internal link on objects
- prototype : property on constructor functions

One-Minute JavaScript Mental Model



"JavaScript is single-threaded, but it can do a lot, thanks to the event loop!"

50 Concepts → One Page (Mind Map)



Before Your JS Interview

- ☐ Explain Event Loop without memorizing
- ☐ Write a closure from scratch
- ☐ Predict async output
- ☐ Explain prototype chain
- ☐ Implement debounce/throttle
- ☐ Explain Promise combinators
- ☐ Identify memory leaks
- ☐ Explain "this" in different contexts
- ☐ Solve array methods (map, filter, reduce)
- ☐ Be comfortable with real world examples

Practice problems > Memorization.
Explain concepts out loud!

Key Takeaways :

- Understand how JavaScript works under the hood.
- Focus on concepts, not just syntax.
- Practice with real world examples.
- Be ready to explain trade-offs and edge cases.
- JavaScript is the foundation of modern web development.

"Don't just write JavaScript. Understand what happens behind the code."

— @abhi_techhub

